



```
def main(src: cown[Account], dst: cown[Account]) {  
    when(src) {  src.balance += 10; }  
    when(dst) {  dst.frozen = true; }  
}
```

```
def main(src: cown[Account], dst: cown[Account]) {  
    when(src) {  src.balance += 10; }  
  
    when(dst) {  dst.frozen = true; }  
  
}
```



```
src.balance += 10;
```



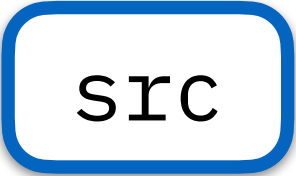


A

when parents and children test themselves it requires

balance: 0

frozen: false



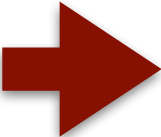
src

balance: 0

frozen: false

dst

BoC: Owns and Behaviours







$\text{dst}.\text{frozen} \equiv \text{true}; \}$

```
def main(src: Account, dst: Account) {
```

when(odstest)





when(sirc) }

src:balance += 10; }



A



B



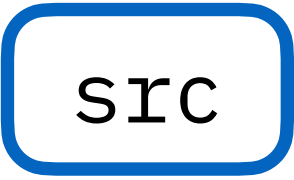


A



balance: 0

frozen: false



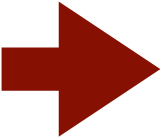
src

balance: 0

frozen: false

dst

Spawning is synchronous but executing is asynchronous



dst: frozen == true; }





